



FINAL YEAR ENGINEERING PROJECT

Smart Medicine Availability & Intelligent Janaushadhi Recommendation System

REACT CONCEPTS

Project	Smart Medicine Availability & Intelligent Janaushadhi Recommendation System
Document	REACT CONCEPTS
Version	v1.0.0
Date	02 July 2026

Learning React Through This Project

1. What is React?

React is a JavaScript library for building user interfaces. Instead of manually updating the HTML DOM whenever data changes, you describe what the UI **should look like** for a given state, and React handles the DOM updates efficiently.

Key idea: React = components + state + re-rendering when state changes.

2. Components

What: A component is a JavaScript function that returns JSX (HTML-like syntax).

Simple example from this project:

```
// src/components/ui/Badge.jsx
function Badge({ children, variant = 'primary' }) {
  return (
    <span className={`badge badge-${variant}`}>
      {children}
    </span>
  )
}
```

Usage:

```
<Badge variant="success">In Stock</Badge>
<Badge variant="danger">Out of Stock</Badge>
```

Why components? Without components, every page would have duplicate HTML for every badge, button, and card. With components, you write once and reuse everywhere.

This project has 50+ components across [components/ui/](#), [components/forms/](#), [components/cards/](#), and more.

3. Props

What: Props (properties) are inputs passed from a parent component to a child component. They are like function parameters.

Example from this project:

```
// SearchResultCard receives props from SearchResultsPage
<SearchResultCard
  medicine={medicineObject}
  layout="grid"
  isBestValue={true}
  onView={() => navigate(`/medicine/${medicine.id}`)}
/>
```

Inside `SearchResultCard`:

```
function SearchResultCard({ medicine, layout, isBestValue, onView }) {
  // medicine, layout, isBestValue, onView are all props
}
```

Key rules:

- Props flow **down** (parent → child)
- Props are **read-only** — a child cannot modify its props
- Use **default props** for optional props: `layout = 'grid'`

4. State (`useState`)

What: State is data that lives inside a component. When state changes, React re-renders the component.

Example from `SearchResultCard`:

```
const [isSaved, setIsSaved] = useState(false)

// Toggle save when user clicks the bookmark button
<button onClick={() => setIsSaved(prev => !prev)}>
  {isSaved ? <HiBookmark /> : <HiOutlineBookmark />}
</button>
```

Example from `AuthContext`:

```
const [currentUser, setCurrentUser] = useState(() => storage.get(STORAGE_KEYS.USER))
const [isLoading, setIsLoading] = useState(false)
const [authError, setAuthError] = useState(null)
```

Why `useState`? It gives React the ability to track changes and re-render only what changed. Without state, the UI would never update.

5. `useEffect`

What: `useEffect` runs code after a component renders. Used for side effects — things that need to happen outside of rendering (API calls, event listeners, timers).

Example from `useOnlineStatus` hook:

```
useEffect(() => {
  const handleOnline = () => setIsOnline(true)
  const handleOffline = () => setIsOnline(false)

  window.addEventListener('online', handleOnline)
  window.addEventListener('offline', handleOffline)

  // Cleanup – remove listeners when component unmounts
  return () => {
    window.removeEventListener('online', handleOnline)
    window.removeEventListener('offline', handleOffline)
  }
}, []) // Empty array = run once on mount
```

Example from `ThemeContext`:

```
useEffect(() => {
  // Runs every time theme changes
  document.documentElement.classList.remove('light', 'dark')
  document.documentElement.classList.add(theme)
  localStorage.setItem(STORAGE_KEYS.THEME, theme)
}, [theme]) // Dependency array: run when theme changes
```

Dependency array rules:

- `[]` — run once after first render
- `[value]` — run after first render AND whenever `value` changes
- no array — run after every render (rarely needed)

6. `useCallback`

What: `useCallback` memoises a function so it is not recreated on every render. Used when a function is passed as a prop or used as a `useEffect` dependency.

Example from AuthContext:

```
const login = useCallback(async (credentials) => {
  setIsLoading(true)
  // ... login logic
}, []) // Empty deps: function never changes
```

Without `useCallback`, the `login` function would be a new object on every render, which could cause infinite loops in `useEffect`.

7. `useRef`

What: `useRef` holds a value that persists between renders but does NOT cause a re-render when changed.

Example from useSessionGuard:

```
const timerRef = useRef(null)

const resetTimer = useCallback(() => {
  clearTimeout(timerRef.current) // Clear old timer
  timerRef.current = setTimeout(...) // Store new timer
}, [])
```

`useRef` is used here instead of `useState` because changing the timer reference should not re-render the component.

8. Context API

What: Context solves the "prop drilling" problem. Instead of passing props through 5 levels of components, you put the data in a Context and any component can access it directly.

How it's used in this project:

```
Provider Tree (main.jsx):
  AuthProvider → provides: currentUser, login, logout
```

ThemeProvider → provides: theme, toggleTheme

Any component anywhere:

```
const { currentUser } = useAuth()      // from AuthContext
const { theme } = useTheme()           // from ThemeContext
```

Creating a context:

```
// 1. Create
const AuthContext = createContext(null)

// 2. Provide
export function AuthProvider({ children }) {
  const [currentUser, setCurrentUser] = useState(null)
  return (
    <AuthContext.Provider value={{ currentUser }}>
      {children}
    </AuthContext.Provider>
  )
}

// 3. Consume
export function useAuth() {
  return useContext(AuthContext)
}

// 4. Use in any component
function Navbar() {
  const { currentUser } = useAuth()
  return <span>{currentUser?.name}</span>
}
```

9. React Router

What: React Router manages navigation in a single-page application (SPA). The URL changes but the page does not reload.

Key hooks used in this project:

Hook	What it does	Example
useNavigate()	Navigate programmatically	navigate('/dashboard')
useLocation()	Get current URL info	location.pathname
useParams()	Get URL parameters	const { id } = useParams()

Route definition (AppRouter.jsx):

```
<Routes>
  <Route path="/" element={<HomePage />} />
  <Route path="login" element={<LoginPage />} />
  <Route path="medicine/:id" element={<MedicineDetailsPage />} />
</Routes>
```

Protected route:

```
// Only authenticated users can access /dashboard
<Route element={<ProtectedRoute />}>
  <Route path="dashboard" element={<UserDashboard />} />
</Route>
```

```
</Route>
```

<Outlet />: In layouts, **<Outlet />** is where the child page renders:

```
function UserLayout() {
  return (
    <div>
      <Sidebar />
      <main>
        <Outlet /> { /* The page renders here */ }
      </main>
    </div>
  )
}
```

10. Lazy Loading

What: Lazy loading delays loading a page's code until the user navigates to it. This makes the initial page load faster.

Used in AppRouter.jsx:

```
// NOT lazy – loads immediately (small, on critical path)
import LoginPage from '../pages/auth/LoginPage'

// LAZY – loads only when user navigates to /dashboard
const UserDashboard = lazy(() => import('../pages/dashboard/UserDashboard'))
```

Why it matters: Without lazy loading, the main bundle would be ~963KB. With lazy loading, it is ~464KB — almost 50% smaller. Users see the home page faster.

11. Custom Hooks

What: A custom hook is a JavaScript function that starts with **use** and uses other React hooks internally. It extracts reusable stateful logic.

Example — useDebounce:

```
// Without the custom hook – duplicated in every search component
const [debouncedValue, setDebouncedValue] = useState(value)
useEffect(() => {
  const timer = setTimeout(() => setDebouncedValue(value), 400)
  return () => clearTimeout(timer)
}, [value])

// With the custom hook – one line anywhere
const debouncedQuery = useDebounce(searchQuery, 400)
```

Custom hooks in this project:

- **useDebounce** — delays search query
 - **useLocalStorage** — persists state to localStorage
 - **useOnlineStatus** — tracks online/offline
 - **useSessionGuard** — monitors inactivity
-

12. Conditional Rendering

What: Showing or hiding elements based on conditions.

Techniques used in this project:

```
// Logical AND (&&) – show only if condition is true
{isBestValue && <div>Best Value</div>}

// Ternary – show A or B
{isLoading ? <Spinner /> : <MedicineList />}

// Early return – return different component
if (!isAuthenticated) {
  return <Navigate to="/login" />
}
return <Dashboard />
```

13. Lists and Keys

What: Rendering lists of data with `.map()`. Each item needs a unique `key` prop.

Example from InventoryPage:

```
{medicines.map((medicine) => (
  <tr key={medicine.id}>
    <td>{medicine.name}</td>
    <td>{medicine.stock}</td>
  </tr>
))}
```

Why keys? React uses keys to track which items changed, were added, or were removed. Without keys, React re-renders the entire list on every change. With keys, it only updates what changed.

14. Forms with React Hook Form + Zod

What: React Hook Form manages form state efficiently. Zod validates form data with a schema.

Example — Login Form:

```
const { register, handleSubmit, formState: { errors } } = useForm({
  resolver: zodResolver(loginSchema)
})

// loginSchema from utils/authSchemas.js:
const loginSchema = z.object({
  email: z.string().email('Enter a valid email'),
  password: z.string().min(1, 'Password is required'),
})

// In JSX:
<form onSubmit={handleSubmit(onSubmit)}>
  <Input
    {...register('email')}
    error={errors.email?.message}
  />
```

```
</form>
```

Why React Hook Form? Traditional form handling with `useState` for every field causes a re-render on every keystroke. React Hook Form uses uncontrolled inputs to avoid this — much better performance.

15. Component Communication Patterns

Pattern 1: Props down, events up

Parent owns the state

- Passes value as prop to Child
- Child fires an event (`onClick`, `onChange`)
- Parent updates state
- Parent passes updated value to Child

Pattern 2: Context (global state)

Any component can read from `AuthContext`

- No need to pass `currentUser` through props

Pattern 3: Local state (within one component)

`isSaved` state in `SearchResultCard`

- Only this card needs to know if it's saved
- No need to lift it to a parent